

Assignment 1 Part B Report Kitty Che 891358920

A. Svelte Component Structure

For Part B, I broke down the original HTML and CSS from Part A into several Svelte components. These include Header.svelte, Footer.svelte, PokemonList.svelte, PokemonDetail.svelte, PokemonEntry.svelte, and TypeDetail.svelte. I mainly divided the components based on their responsibilities so that each component handles a clear and relatively independent part of the website.

Header.svelte handles the title and menu at the top of the website, including the background music, theme selection, and user guide. Footer.svelte contains the information displayed at the bottom of the website. Since the Header and Footer are shared parts that should always be displayed, I placed them in +layout.svelte rather than on the individual page.

The main content is divided into PokemonList.svelte and PokemonDetail.svelte. PokemonList handles the Pokémon list, search function, generation selection, and Pokémon selection. PokemonDetail displays information about the currently selected Pokémon, including its image, basic information, shiny form, cry sound, and evolution information. Since these two components need to share the currently selected Pokémon, I store selectedPokemon in their parent component, +page.svelte. PokemonList can update this state when the user selects a Pokémon, while PokemonDetail receives it and displays the corresponding information.

I also separated the Pokédex entry and Pokémon type information into PokemonEntry.svelte and TypeDetail.svelte. TypeDetail is also responsible for the Type Effectiveness feature that I added. It has its own data fetching, user interaction, and CSS, so separating it from PokemonDetail helps prevent the detail component from becoming too large and keeps the responsibilities of each component clearer.

For CSS, most styles that only apply to a specific component are kept inside the <style> section of that Svelte component. For example, the list layout and scrollbar are styled inside PokemonList.svelte, while the Pokémon images, information cards, and animations are styled inside PokemonDetail.svelte. I still use app.css for styles that need to apply across multiple components. This includes the general body styling, custom fonts, and the theme system. Since changing the theme affects the Header, Pokémon list, detail section, type information, and Footer at the same time, keeping these rules in the global CSS avoids repeating the same theme styling across several components.

B. Fetching Data

My application fetches data mainly within PokemonList.svelte, PokemonDetail.svelte, and TypeDetail.svelte. I decided where to place each fetch based on which component is responsible for using the data. PokemonList.svelte fetches the Pokémon list. The generation selection, search function, and Pokémon selection are all handled by this component, so keeping the list fetching here allows these related functions to stay together.

When the user selects a Pokémon, `PokemonDetail.svelte` uses its `dexNumber` to fetch its detailed information. I also use `PokéAPI` to fetch additional Pokémon and species information and the evolution chain. This provides the extra information used in the detail section, such as height, weight, gender, category, and evolution. Since this information is only needed in the Pokémon detail section, I decided to keep this fetching logic inside `PokemonDetail`.

`TypeDetail.svelte` separately fetches the `damage_relations` for each Pokémon type. This information is only used by my Type Effectiveness feature, so I decided that `TypeDetail` should fetch and manage this data itself instead of fetching it in the parent component and passing it down.

C. Extra Features

Generation Filtering

I created `selectedGen` as a `$state` variable with "all" as its default value. The generation `<select>` is connected to it using `bind:value={selectedGen}`, so changing the selected option automatically updates `selectedGen`. I use `$effect` to call `fetchPokemonList(selectedGen)`. Since `selectedGen` is used inside the effect, the effect runs again whenever its value changes. The selected generation is added to the API request as a query parameter such as `?gen=1` or `?gen=5`. As a result, changing the generation sends a new request to the server and retrieves a new Pokémon list instead of filtering the original list only on the client side. The search function uses a separate `typedSearch` state and `$derived` to create `filteredPokemons`. Because the search text is stored separately from the generation selection, the current search is still applied to the newly fetched list when the user changes generation.

Pokémon Cry Sound

Users can play a Pokémon's cry by clicking on its image. The detailed Pokémon data provided by the assignment API contains the `crySound` URL, which I use when the image is clicked. The image has an `onclick` event that calls my `playCry()` function. Inside this function, I use the browser's JavaScript Audio API to create an audio object with `new Audio(pokemon.crySound)`. I then call `.play()` to play the sound. I also set an `isShake` state to true when the cry is played. This applies a CSS shake animation to the Pokémon image and gives the user visual feedback at the same time as the sound. When the animation finishes, the state is reset so the animation can run again the next time the image is clicked.

Unique Feature

My unique feature is an interactive Type Effectiveness system. It allows users to see more than just the Pokémon's type by showing how each type performs in both offensive and defensive situations. `TypeDetail.svelte` uses the Pokémon's type to request additional `damage_relations` data. For a dual-type Pokémon, the application fetches the information for both types. When the user hovers over a type button, a `mouseenter` event updates `hoveredType`. The component then displays the information that matches the type being hovered over. I separate the data into Offense and Defense sections. Relations containing “_to” are used for offensive information such as Strong against, Resisted by, and

No effect on. Relations containing “_from” are used for defensive information such as Weak to, Resists, and Immune to. The related types are displayed using their type icons to make the information easier to read. For Pokémon with two types, users can hover over each type separately to view its individual effectiveness.

D. GenAI Usage

I mainly used ChatGPT with the GPT-5.6 Sol model as a learning and debugging tool while developing this website.

One of the most helpful uses was understanding the structure and direction of data flow in Svelte. After breaking the original HTML into different components, I was initially unsure about which component should store certain data and how data should be passed between parent and child components. ChatGPT helped me understand how my components work together and the roles of \$props, \$bindable, and shared state. This gave me a clearer understanding of how data flows through the application. ChatGPT also helped me understand and visualise the structure of data returned by APIs. Some API responses contain multiple levels of nested objects and arrays, such as the evolution chain and type damage relations. Breaking these structures down into simpler levels made it easier for me to understand which properties I needed to access and how different parts of the data were related.

Another helpful use was debugging. During development, there were situations where a small change caused content that previously worked to disappear or an interaction to stop working. ChatGPT helped me narrow down the cause of these problems more quickly. For example, I originally used `selectedPokemon === pokemon` to determine which Pokémon in the list was selected. After adding generation filtering and fetching a new Pokémon list, the selected highlight could disappear. ChatGPT helped me identify that the newly fetched list contained new Pokémon objects. I therefore changed the comparison to use `dexNumber`, with `selectedPokemon?.dexNumber === pokemon.dexNumber`, to determine whether they represented the same Pokémon. ChatGPT also helped me decide which data should be checked when displaying the loading page. `PokemonDetail` depends on several pieces of data, including Pokémon details, card details, species details, and the evolution chain. I needed to make sure the required data had loaded before displaying the full content. By working through the dependencies between these pieces of data, I was able to create a clearer loading condition and avoid trying to access properties while the data was still null.

One limitation I found when using GenAI was that the quality of its answers depended heavily on the context I provided. Many problems in my application involved multiple components, states, or CSS files. If I only provided one part of the code, ChatGPT could make suggestions based on incomplete information. A solution could therefore seem reasonable on its own but fail to solve the problem when applied to the full application or sometimes affect another feature that was already working. Because of this, I needed to provide enough relevant code and understand how my components worked together before deciding whether an AI suggestion was suitable.